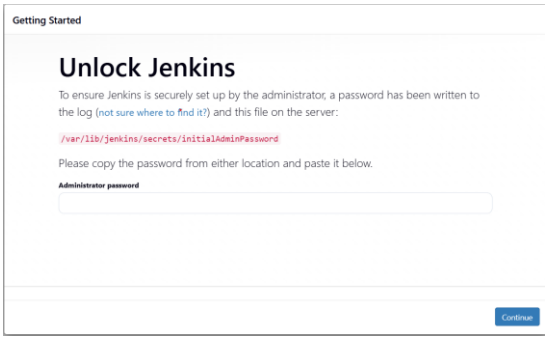
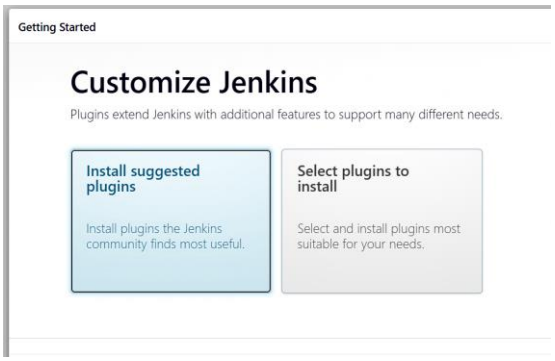
Steps for Integration of GIT with Jenkins

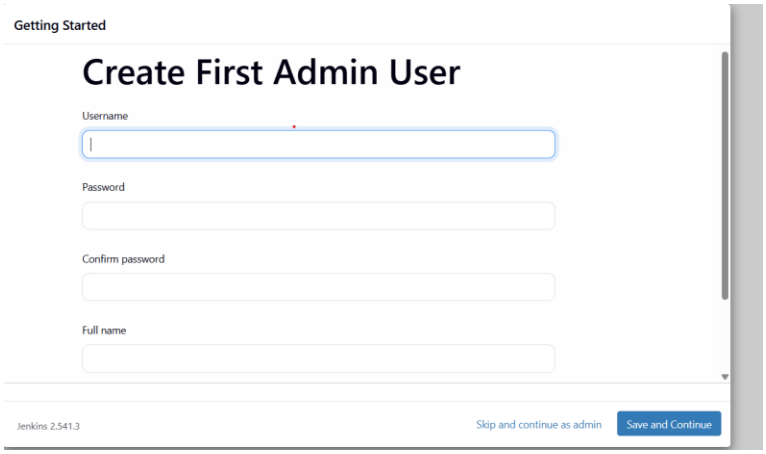
1. Create a EC2 instance and connect to the EC2 instance & login as route user.
2. Install the Jenkins by using below commands:
   yum install java-17-amazon-corretto -y
   sudo wget -O
   /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/rpm-stable/jenkins.repo
   yum install jenkins -y
   systemctl start jenkins
   systemctl status Jenkins
3. Copy the public ip of your server and paste it in your browser followed by port number 8080:
   Ex: http://52.204.101.48:8080/
4. After accessing url you will get below screen copy the path and enter cat /path in your terminal then you get token copy and paste it in Jenkins browser.



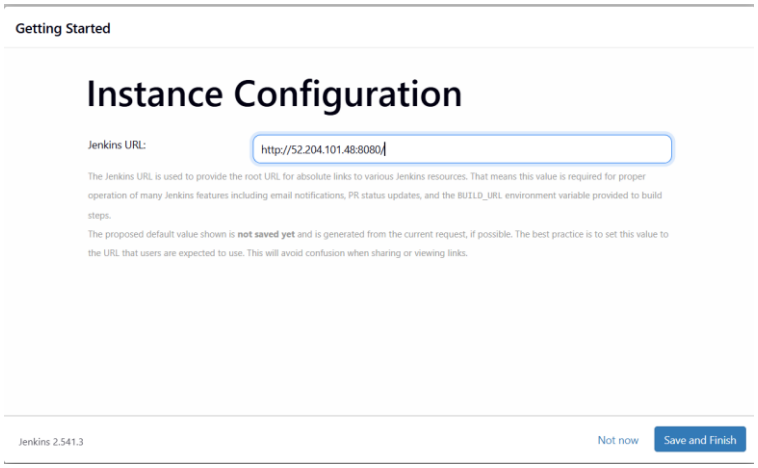
5. Click on Install suggested plugins and pugins will install.



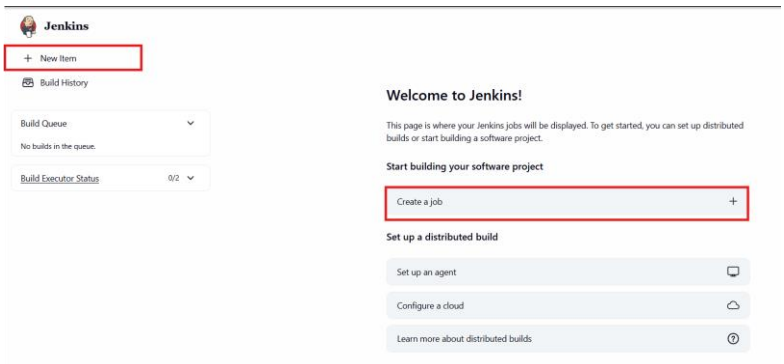
6. Create user details for login



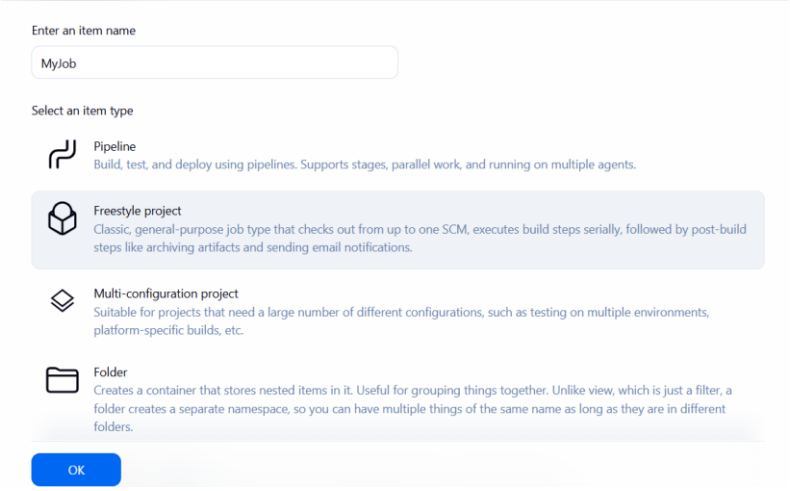
7. Click Save and Finish



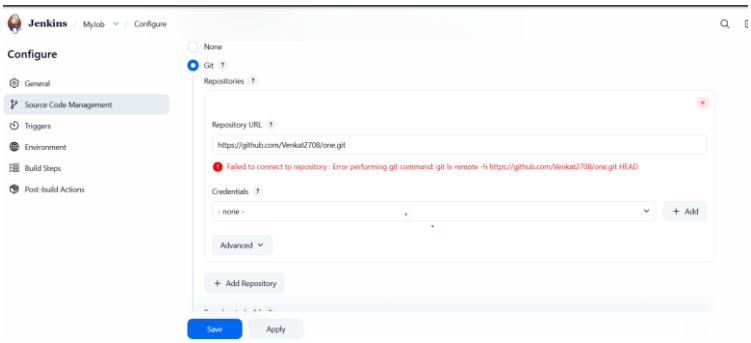
8. Create your Job or pipeline by clicking New item or Create a job.



9. Enter Item name and select Freestyle project and Click ok

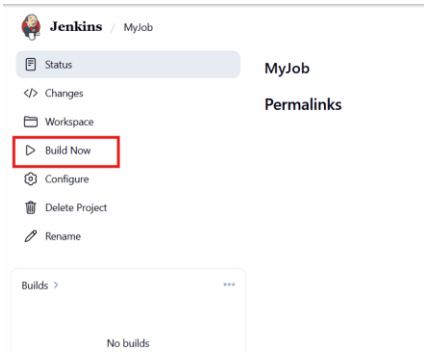


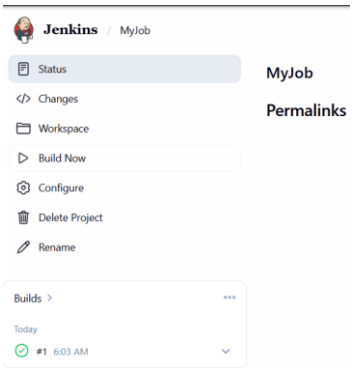
10. Copy the GIT repo URL and goto your Jenkins dashboard and click on General tab paste the repo URL and click on save.



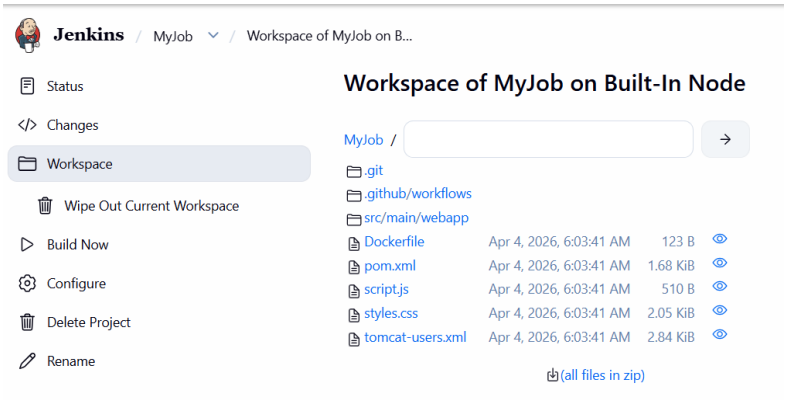
11. If you get above error then git is not installed in your server. Goto your terminal and install git by using below command.
yum install git -y

12. Click on build now to get your files in Jenkins dashboard

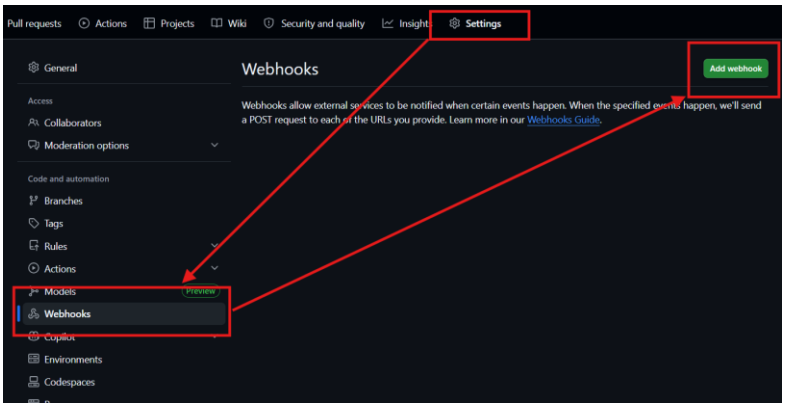




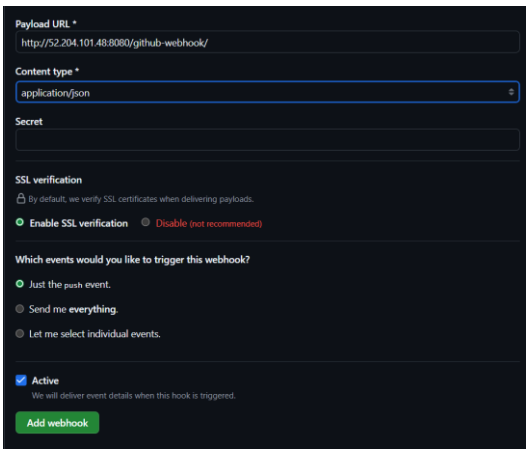
13. Click on workspace to see the files.

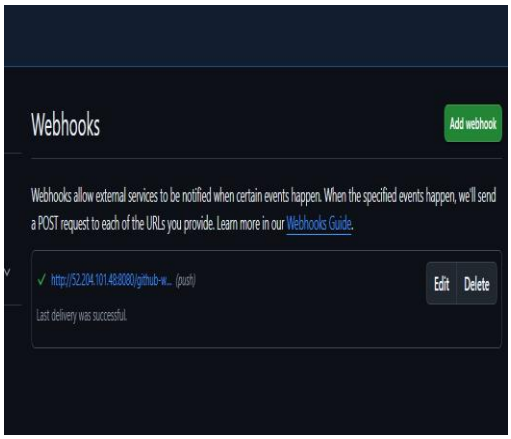


14. Instead of manual Build we can automate when we perform any commit on GITHUB by using webhook.
15. Goto your GITHUB and Click on repo settings and Click on Webhooks and Click on Add webhook



16. Enter payload URL as your Jenkins URL till 8080 and after that enter github-webhook/ and select content type as application/json and Click on Add webhook. If you see Last delivery was successful then your Payload URL is successfully configured.





17. Goto Configure in Jenkins Dashboard and Click on Triggers and check the box **GitHub hook trigger for GITScm polling and Click Save.**
18. Goto GITHUB and perform one commit and then Build will perform automatically in your Jenkins dashboard

Poll SCM: -

Poll SCM in Jenkins is a build trigger that periodically checks your source code repository for changes, and if new commits are found, Jenkins automatically starts a build. It's configured using cron-like syntax and is often used as a fallback when webhooks aren't working.

Configuration of Poll SCM:

1. **Open Jenkins Job Configuration**:
   - Go to your job → **Configure**.
2. **Enable Poll SCM**:
   - Under **Build Triggers**, check **Poll SCM**.
3. **Set Schedule (Cron Syntax)**:

   *** * * * ***

   - Example:
     - H/5 * * * * → Poll every 5 minutes.
     - H 2 * * * → Poll once daily at 2 AM.
     - H/15 * * * * → Poll every 15 minutes.
   - H distributes load evenly across Jenkins nodes.
4. **Save Configuration**:
   Jenkins will now automatically check the repository at the defined intervals

Build Periodically:

Definition: It schedules builds at specific times using cron-like syntax.

Trigger: Jenkins runs the job at the scheduled time, even if no code changes have occurred.

Use Case: Useful for nightly builds, scheduled deployments, or routine tasks like running tests or generating reports.

Configuration of Bulid Periodically:

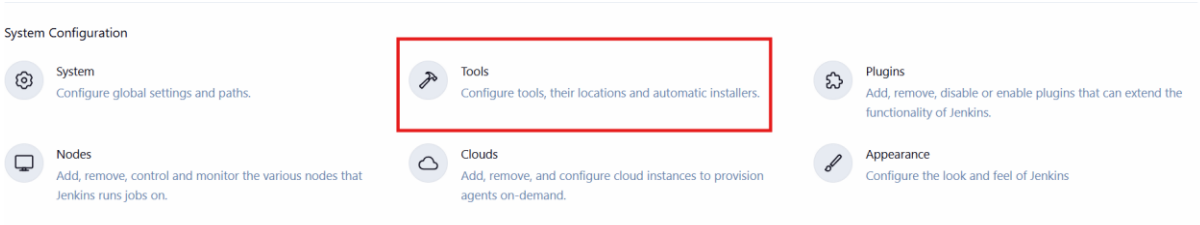
1. Go to your Jenkins job → **Configure**.
2. Under **Build Triggers**, check **Build periodically**.
3. Enter a schedule using cron syntax:
    - H/30 * * * * → Run every 30 minutes.
    - H 2 * * * → Run daily at 2 AM.
    - H 0 * * 0 → Run every Sunday at midnight.
4. Save the job. Jenkins will now trigger builds automatically at those times.

MAVEN

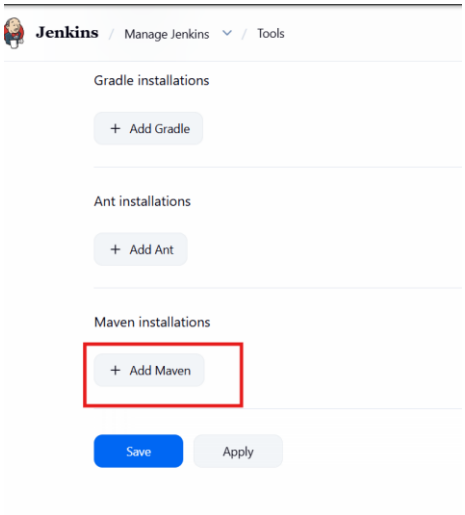
Apache Maven is a powerful build automation and project management tool for Java projects. It uses a standardized project structure and a central configuration file (pom.xml) to handle compilation, testing, packaging, and dependency management.

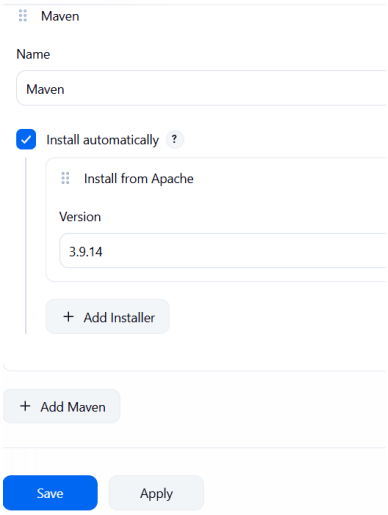
Integration of Maven with Jenkins:

1. Go To Manage Jenkins and Click on Tools

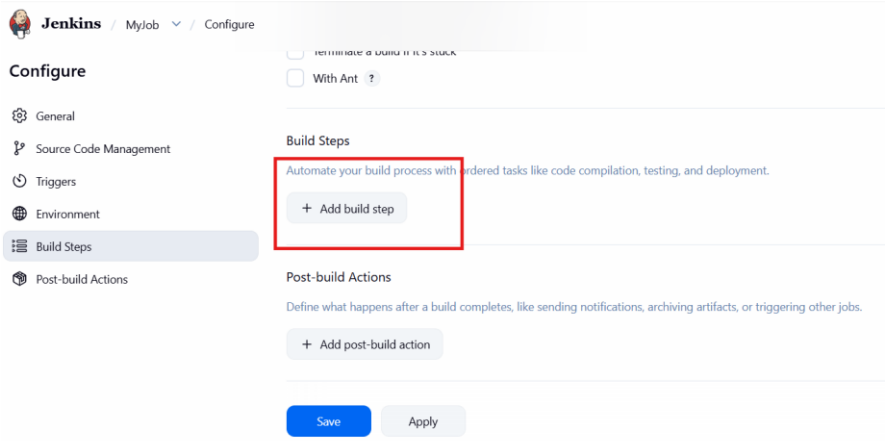


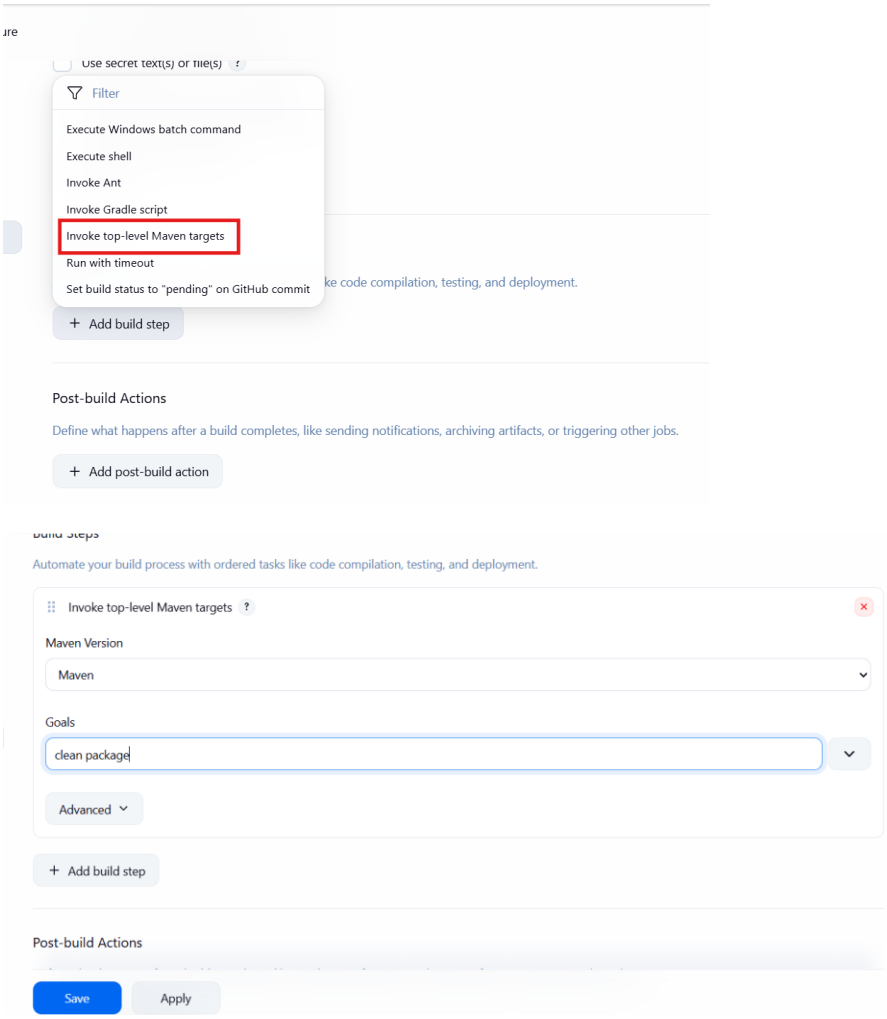
2. Under Maven Installation Click on Add Maven and Enter Name and Click on Save.





3. Then GOTO your Job and Click on Configure and Under Build Steps Click on Add build step → Select Invoke top-level Maven targets → Select Maven Version as your name i.e. created under Maven Installation → Give goal as command → Click Save





4. Click on Build now and it will generate war file and you can see your file workspace (target folder)

Integration of Tomcat Server with Jenkins

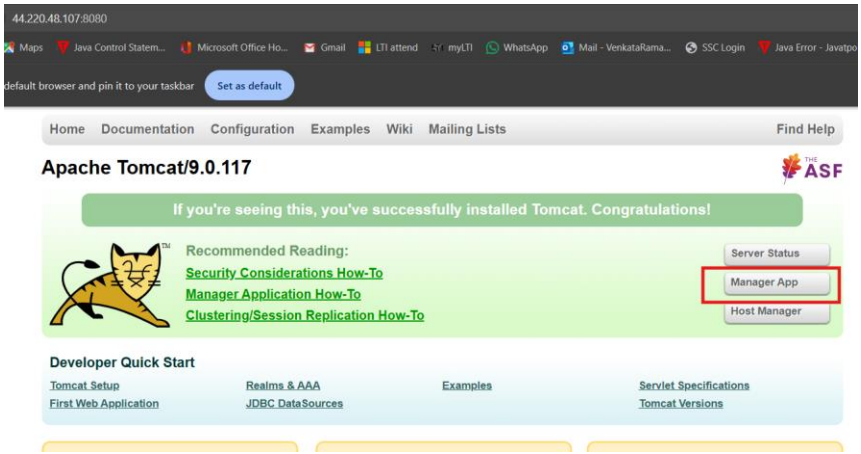
1. Create an EC2 Instance in AWS server.
2. Login with root user.
3. Install the JAVA and Tomcat server and Start the tomcat server by following command:

   ```
   yum install java-17-amazon-corretto -y
   wget https://dlcdn.apache.org/tomcat/tomcat-9/v9.0.117/bin/apache-tomcat-9.0.117.tar.gz
   tar -zxvf apache-tomcat-9.0.117.tar.gz
   sed -i '56  a\<role rolename="manager-gui"/>' apache-tomcat-9.0.117/conf/tomcat-users.xml
   sed -i '57  a\<role rolename="manager-script"/>' apache-tomcat-9.0.117/conf/tomcat-users.xml
   sed -i '58  a\<user username="tomcat" password="admin@123" roles="manager-gui, manager-script"/>' apache-tomcat-9.0.117/conf/tomcat-users.xml
   sed -i '59  a\</tomcat-users>' apache-tomcat-9.0.117/conf/tomcat-users.xml
   sed -i '56d' apache-tomcat-9.0.117/conf/tomcat-users.xml
   sed -i '21d' apache-tomcat-9.0.117/webapps/manager/META-INF/context.xml
   sed -i '22d'  apache-tomcat-9.0.117/webapps/manager/META-INF/context.xml
   sh apache-tomcat-9.0.117/bin/startup.sh
   ```

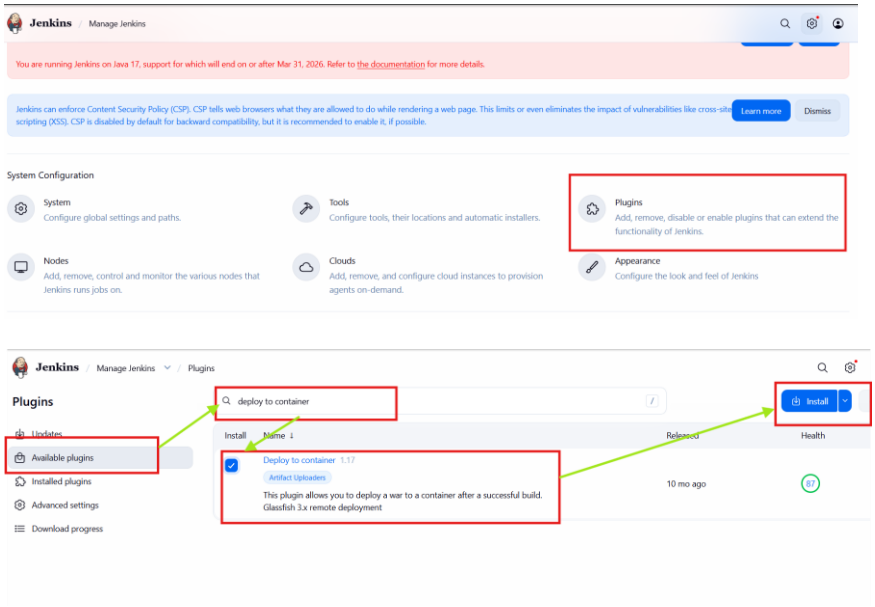
4. After tomcat setup is done. Access the application using Public IP followed by :8080 and login with your credentials.

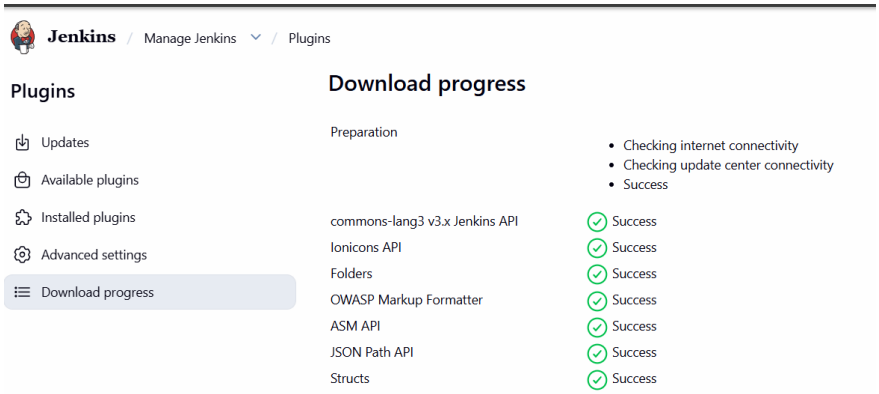
   Ex: http://44.220.48.107:8080/

5. Goto Manager APP to check the tomcat configuration:

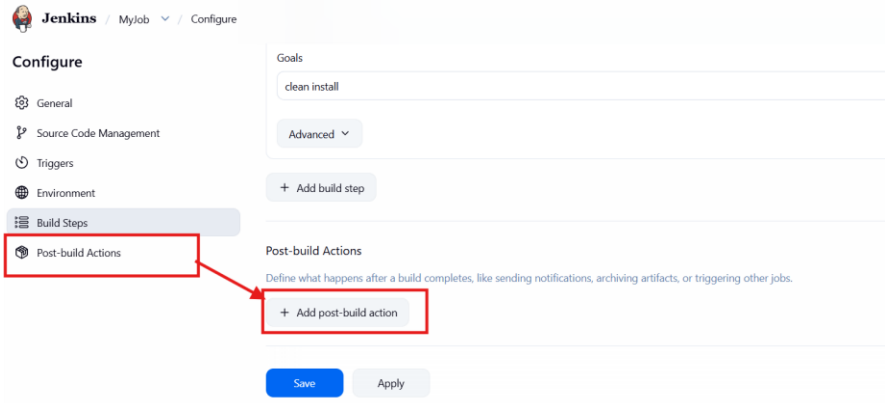


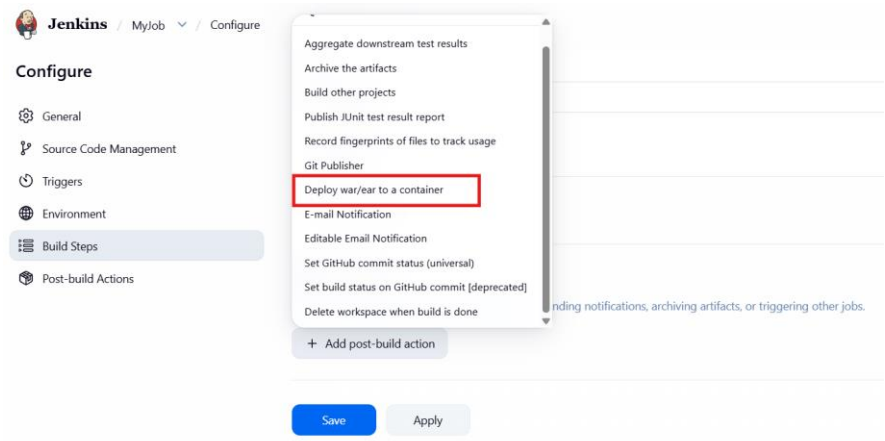
6. Then, GOTO your Jenkins application and GOTO Manage Jenkins and Click on plugins → Available plugins → Search for Deploy to container → Select and Click on Install



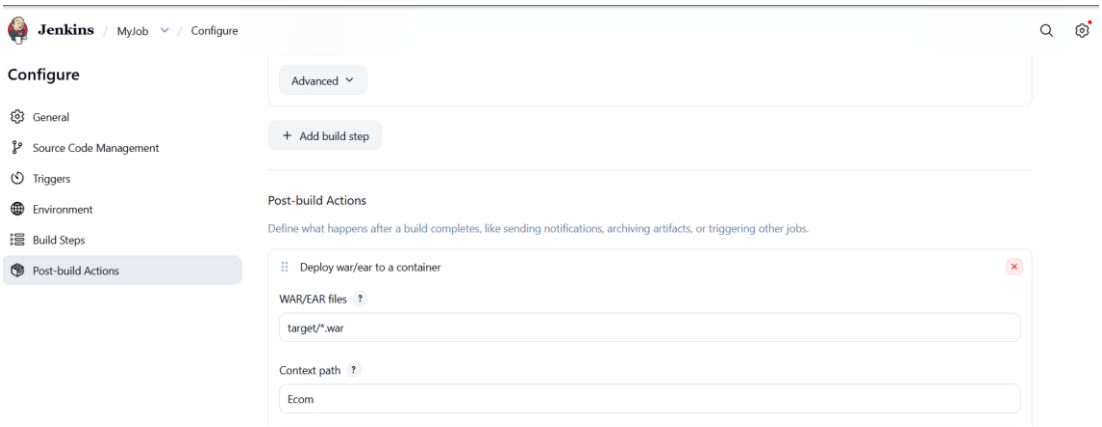


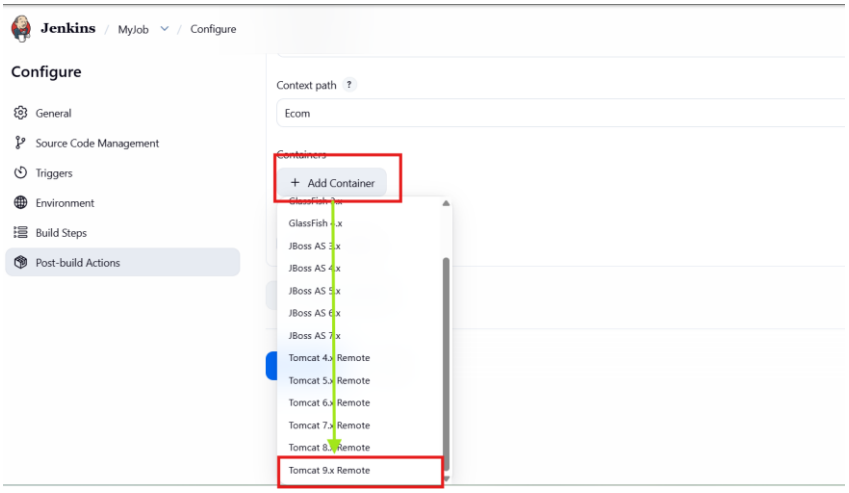
7. Open your pipeline & Click on Configure and Click on Post-build Actions → Click on Add post-build action → Select **Deploy war/ear to a container** from dropdown



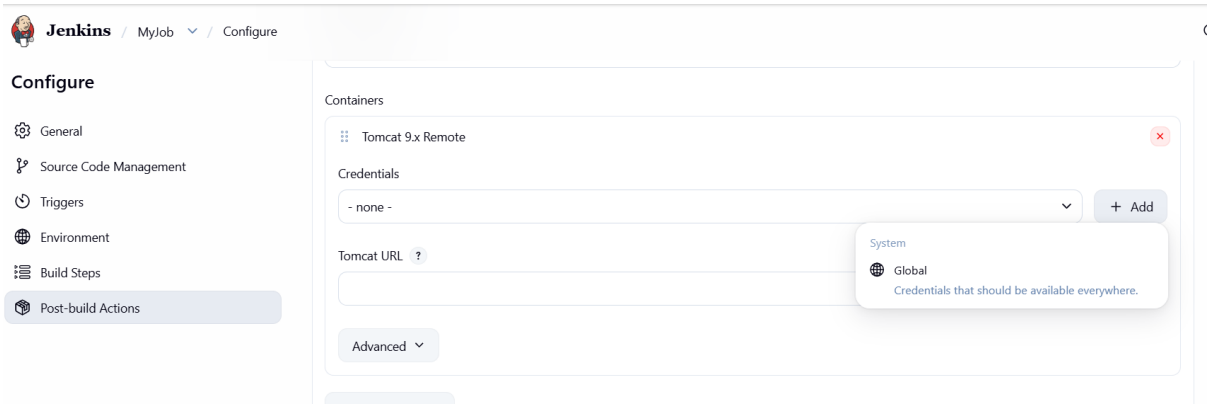


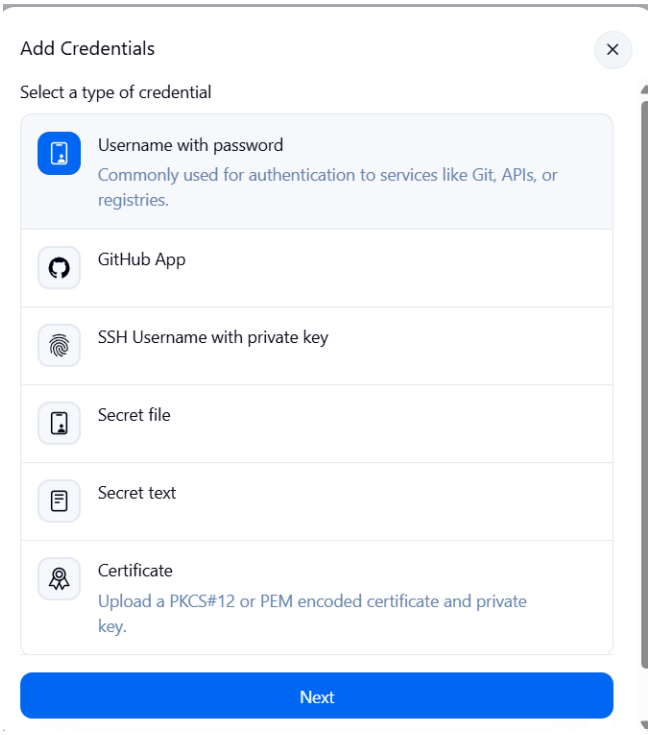
8. Give the **WAR/EAR files as target/*.war** & Context path as **Application name** → Click on Add Container and select the Tomcat 9.x Remote from dropdown.



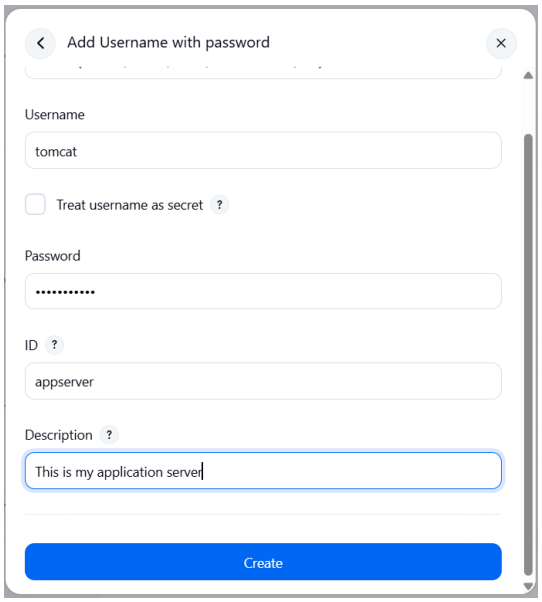


9. Click on +Add and Click on Global & Select Username with password → Next

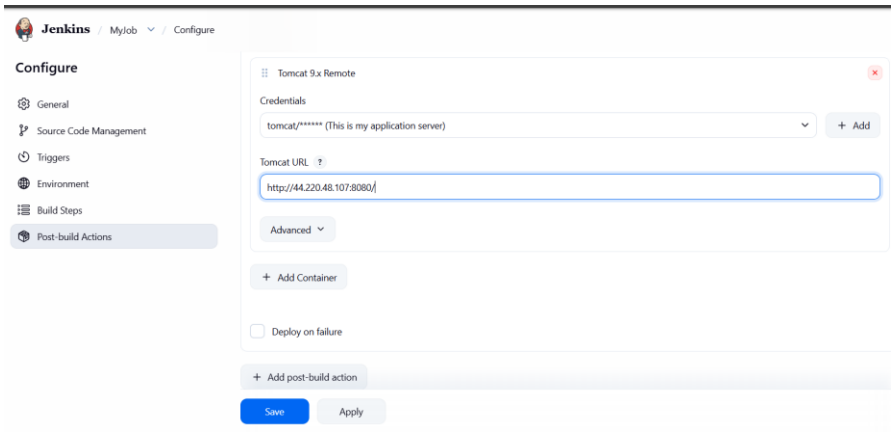
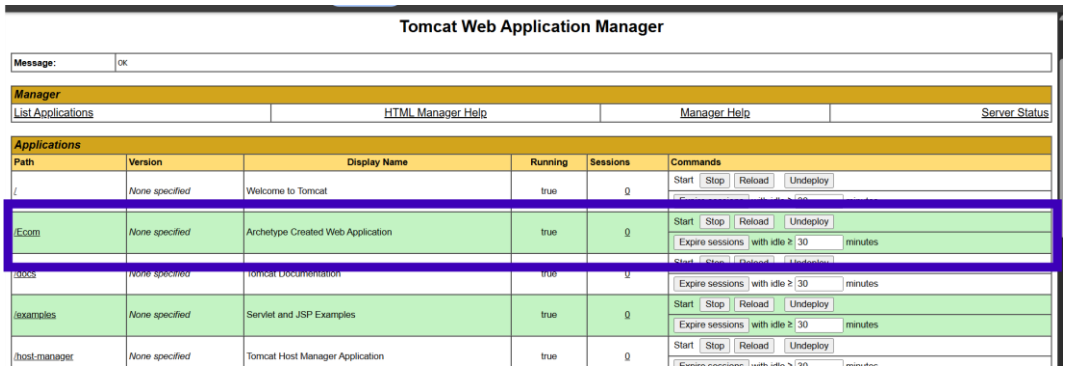
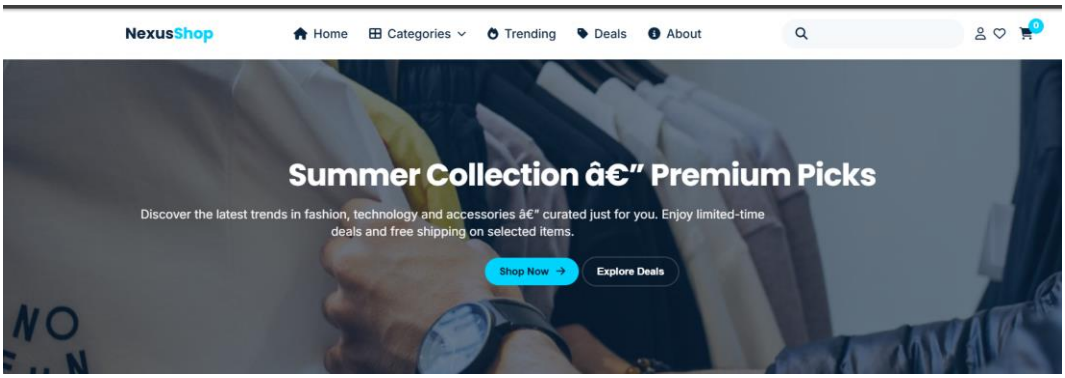




10. Give your tomcat Username and Password & Give ID as your wish (In my case I am giving appserver and Give Description and Click Create.



11. Select Credentials that are created in previous step and Give the Tomcat URL followed by :8080 → Click Save



12. Click on Build Now and Goto your tomcat page →Manager app→ You will find your application → Click on the application to test